COURSE-BASED PROJECT DOCUMENTATION | DOC-06

Complete Execution Workflow

Internal Execution Trace, CLI Flow, & GUI Interactive Pipelines

1. High-Level Pipeline (Viva Explanation)

The complete execution path from source file to graphical highlight proceeds as follows:

VIVA ONE-SENTENCE PIPELINE
Java source file -> Flex lexer -> Token stream -> Bison LALR(1) parser -> Shift/Reduce actions -> Syntax result -> javacheck.dll / javacheck.exe -> Python ctypes backend -> Tkinter GUI -> Result / error-line highlighting

2. Internal C Engine Execution Trace

- 1. check_syntax(filepath, err_msg, len) is called.
- 2. State variables are reset: syntax_error_occurred = 0, error_line = 0, yylineno = 1, current_column = 1, yylloc = {1,1,1,1}.
- 3. File handle is opened: FILE *f = fopen(filepath, 'r'). If NULL, returns 2 (I/O error).
- 4. Scanner buffer is re-initialized: yyrestart(f); yyin = f;.
- 5. Parser loop begins: yyparse() is invoked.
- 6. yyparse() repeatedly calls yylex() to fetch lookahead tokens.
- 7. YY_USER_ACTION updates yylloc coordinates before every token action.
- 8. Parser shifts tokens or reduces rules against LALR(1) tables.
- 9. On syntax error, Bison calls yyerror(); error_line is extracted from yylloc.first_line.
- 10. check_syntax() closes file handle and formats result string.

3. CLI vs. GUI Workflow Comparison

Workflow Step	CLI Execution (javacheck.exe)	GUI Execution (gui/app.py)
Trigger	User runs command in shell	User presses Ctrl+Enter or clicks...
Input Source	Disk file path from argv[1]	In-memory editor text or opened file
Invocation	Direct C main() execution	In-memory ctypes CDLL invocation
Success Feedback	stdout: SUCCESS: Valid Java Syntax	Green status bar & clean editor
Error Feedback	stderr: ERROR: Line X: Syntax err...	Red status bar, gutter color, lin...